

BTS SERVICES INFORMATIQUES AUX ORGANISATIONS	SESSION 2026
ANNEXE VII-1-B : Fiche descriptive de réalisation professionnelle (recto)	
Épreuve E6 - Conception et développement d'applications (option SLAM)	

DESCRIPTION D'UNE RÉALISATION PROFESSIONNELLE		N° réalisation : 1
Nom, prénom : Spirine Vladimir		N° candidat : 2545863387
Épreuve ponctuelle ☒	Contrôle en cours de formation ☐	Date : 08/06/2026
Organisation support de la réalisation professionnelle SYADEM		
Intitulé de la réalisation professionnelle Outils d'analyse de tickets		
Période de réalisation : Décembre 2024 – Mars 2026 Lieu : Bordeaux		
Modalité : ☒ Seul(e) ☐ En équipe		
Compétences travaillées ☒ Concevoir et développer une solution applicative ☒ Assurer la maintenance corrective ou évolutive d'une solution applicative ☒ Gérer les données		
Conditions de réalisation¹ (ressources fournies, résultats attendus) Ressources fournies : Exports de tickets issus de la plateforme Zendesk au format XML. Résultats attendus : Mettre en place un outil capable d'importer, nettoyer, structurer et analyser les tickets afin d'identifier les problèmes récurrents, faciliter leur exploitation et produire des résultats utiles sous forme de regroupements, résumés et statistiques.		

¹ En référence aux *conditions de réalisation et ressources nécessaires* du bloc « Conception et développement d'applications » prévues dans le référentiel de certification du BTS SIO.

Description des ressources documentaires, matérielles et logicielles utilisées

Ressources documentaires :

Documentation officielle Ruby ; documentation Nokogiri (XML/HTML) ; documentation API Zendesk v2 ; documentation Ollama (API locale pour LLM) ; documentation Git ; échanges avec les utilisateurs métier.

Ressources matérielles :

Ordinateur de développement dédié au projet ;

Environnement local de test et d'exécution ;

Poste configuré pour les traitements de données et l'exécution locale des modèles.

Ressources logicielles (versions utilisées) :

- Ruby 3.3.10p183 ;
- Bundler 2.5.23 ;
- Nokogiri 1.19.1 ;
- HTTParty 0.24.2 ;
- dotenv 3.2.0 ;
- numo-narray 0.9.2.1 ;
- Rumale 2.1.0 (algorithmes de clustering, dont KMeans) ;
- Ollama (client 0.18.0) comme API locale pour piloter les LLM ;
- API Zendesk v2 ;
- Git 2.49.0.windows.1 ;
- Visual Studio Code 1.115.0.

Modalités d'accès aux productions et à leur documentation

Code source :

Dépôt GitHub du projet : https://github.com/Farounaga/rb_tkts

Documentation du projet :

Dossier documentaire partagé :

https://drive.google.com/drive/folders/1ZjLn7umvwE_GNQ66TI6o1gSbzg24q19d?usp=drive_link

Documents disponibles :

- Schéma de base de données
- Cahier des spécifications / cahier des charges
- Cahier de tests
- Documentation technique
- Documentation utilisateur
- Guides d'exploitation

**ANNEXE VII-1-B : Fiche descriptive de réalisation professionnelle
(verso, éventuellement pages suivantes)**

Épreuve E6 - Conception et développement d'applications (option SLAM)

Descriptif de la réalisation professionnelle, y compris les productions réalisées et schémas explicatifs

Contexte

Cette réalisation s'inscrit dans le cadre de mon alternance au sein de **SYADEM**, éditeur de solutions logicielles dédiées à la vaccination et à la prévention. En tant que jeune développeur intégré à l'équipe de développement, cette mission m'a été confiée afin de me permettre de monter progressivement en compétences sur un besoin concret de l'entreprise.

La mission portait sur le traitement et l'analyse d'un volume important de tickets issus de l'activité support, un domaine nécessitant rigueur, fiabilité et autonomie.

Objectif de la réalisation

Concevoir et développer un outil Ruby d'analyse de tickets permettant d'automatiser la chaîne de traitement, depuis l'import des données jusqu'à la restitution d'indicateurs utiles aux équipes support et produit.

Méthodologie de travail mise en œuvre

- Recueil et clarification du besoin métier avec les utilisateurs ;
- Découpage en modules (import, parsing, nettoyage, vectorisation, clustering, similarité, reporting) ;
- Développement itératif avec tests réguliers sur des exports réels ;
- Versioning Git et validation progressive des résultats ;
- Ajustements des paramètres d'analyse selon les retours métier.

Architecture de la solution

- Source de données : exports de tickets Zendesk (XML) ;
- Couche de traitement : scripts Ruby + Nokogiri pour l'extraction et la normalisation ;
- Couche IA locale : Ollama utilisé comme API locale pour piloter les LLM servant à la vectorisation et à la génération de résumés thématiques ;
- Couche analytique : clustering KMeans + calcul de similarité cosinus ;
- Couche de restitution : fichiers de synthèse et rapports exploitables par les équipes.

Modalités de déploiement

La solution est exécutée en environnement local/interne, avec une configuration centralisée (variables d'environnement), des scripts versionnés et des sorties intermédiaires conservées pour audit et comparaison des exécutions.

Déploiement

Exécution en environnement local/interne via scripts Ruby versionnés et configuration .env.

Sécurité

Secrets/API keys hors dépôt Git, accès interne contrôlé, minimisation des données traitées.

Test

Tests automatiques (rspec) et manuels, validations fonctionnelles sur exports réels, contrôle des clusters/similarités/résumés, validation métier.

Productions réalisées :

- Développement du pipeline complet d'analyse des tickets ;
- Structuration et normalisation des données textuelles ;
- Mise en place des traitements IA/clusterisation ;
- Génération de livrables de synthèse (groupements, statistiques, éléments d'aide à la décision).

Flux de traitement (étapes) :

- Import des exports de tickets Zendesk (XML)
- Extraction et parsing des données (Ruby + Nokogiri)
- Nettoyage et normalisation des tickets

- Vectorisation des contenus (embeddings via Ollama)
- Clustering KMeans et calcul de similarité cosinus
- Génération de résumés thématiques et reporting
- Restitution des résultats pour les équipes support et produit

BTS SERVICES INFORMATIQUES AUX ORGANISATIONS	SESSION 2026
ANNEXE VII-1-B : Fiche descriptive de réalisation professionnelle (recto)	
Épreuve E6 - Conception et développement d'applications (option SLAM)	

DESCRIPTION D'UNE RÉALISATION PROFESSIONNELLE		N° réalisation : 2
Nom, prénom : Spirine Vladimir		N° candidat : 2545863387
Épreuve ponctuelle ☒	Contrôle en cours de formation ☐	Date : 08/06/2026
Organisation support de la réalisation professionnelle Campus Ermitage		
Intitulé de la réalisation professionnelle Site de rencontre E-Sport (GameConnect)		
Période de réalisation : 19/01/2025 – 25/09/2025 Lieu : Agen Modalité : ☐ Seul(e) ☒ En équipe		
Compétences travaillées ☒ Concevoir et développer une solution applicative ☐ Assurer la maintenance corrective ou évolutive d'une solution applicative ☒ Gérer les données		
Conditions de réalisation¹ (ressources fournies, résultats attendus) Ressources fournies : Environnement de développement avec Visual Studio Code, technologies front-end React et Tailwind CSS, environnement back-end Python avec FastAPI, base de données MySQL, ainsi que les éléments nécessaires à la conception et au développement de l'application. Résultats attendus : Mettre en place une application de mise en relation entre joueurs, créer une base de données permettant de stocker les informations liées aux jeux et aux utilisateurs, et permettre les interactions ainsi que la mise en relation entre différents joueurs.		

¹ En référence aux *conditions de réalisation et ressources nécessaires* du bloc « Conception et développement d'applications » prévues dans le référentiel de certification du BTS SIO.

Description des ressources documentaires, matérielles et logicielles utilisées

Ressources documentaires :

Documentation React/Tailwind/Vite pour l'interface ; documentation FastAPI et Pydantic pour les endpoints REST ; documentation MySQL pour la modélisation et les requêtes ; documentation Git ; notes techniques du projet (architecture, API, tests).

Ressources matérielles :

- Ordinateur de développement ;
- Environnement local de test et d'exécution ;
- Poste configuré pour l'exécution du front, de l'API et de la base de données.

Ressources logicielles (versions utilisées) :

- Visual Studio Code 1.115.0 ;
- Node.js v24.13.1 et npm 11.2.0 ;
- React 18.2.0, React DOM 18.2.0, React Router DOM 6.8.0 ;
- Tailwind CSS 3.4.17 ;
- Vite 5.0.8 ;
- Axios 1.6.0 ;
- FastAPI 0.104.1 ;
- Uvicorn[standard] 0.24.0 ;
- Python 3.13.3 (projet compatible Python 3.8+) ;
- PyJWT 2.8.0 ;
- bcrypt 4.1.1 ;
- pydantic[email] 2.3.0 et email-validator 2.1.0.post1 ;
- python-dotenv 1.0.0 ;
- requests 2.32.3 ;
- MySQL Server 8.0.40 et mysqlclient 2.2.0 ;
- Git 2.49.0.windows.1.

Modalités d'accès aux productions et à leur documentation

Code source :

Dépôt GitHub du projet : <https://github.com/Farounaga/Esportapp>

Documentation du projet :

Dossier documentaire partagé :

https://drive.google.com/drive/folders/1ZjLn7umvwE_GNQ66TI6o1gSbzig24q19d?usp=drive_link

Documents disponibles :

Schéma de base de données
Documentation technique API
Cahier de tests
Documentation utilisateur
Notes de déploiement et d'exploitation

**ANNEXE VII-1-B : Fiche descriptive de réalisation professionnelle
(verso, éventuellement pages suivantes)**

Épreuve E6 - Conception et développement d'applications (option SLAM)

Descriptif de la réalisation professionnelle, y compris les productions réalisées et schémas explicatifs

Contexte

La réalisation a été menée dans le cadre de ma formation BTS SIO option SLAM au Campus Ermitage, en équipe de trois étudiants, où je me suis occupé de l'API. Le projet répond à la demande d'un commanditaire fictif, l'entreprise DevDuJour, spécialisée dans l'organisation d'événements e-sport et le coaching de joueurs. Le besoin exprimé est de faciliter la mise en relation de joueurs compatibles : les gamers perdent aujourd'hui un temps considérable à chercher des coéquipiers partageant les mêmes jeux, niveaux de compétence et disponibilités sur des forums et réseaux sociaux dispersés. L'objectif était de concevoir une plateforme web centralisée, GameConnect, permettant d'automatiser la recherche de coéquipiers via un algorithme de compatibilité, de faciliter la communication entre joueurs matchés et de sécuriser les échanges et les données personnelles.

Objectifs du projet

- Mettre en relation des joueurs compatibles selon leurs jeux, niveaux et disponibilités ;
- Faciliter la communication entre joueurs matchés pour organiser leurs sessions ;
- Créer une communauté active autour des jeux via des espaces d'échange ;
- Permettre la gestion de l'identité gaming (profil, préférences, statistiques) ;
- Automatiser la recherche de coéquipiers via un algorithme de compatibilité ;
- Sécuriser les données personnelles et les échanges entre utilisateurs.

Fonctionnalités sur lesquelles j'ai travaillé

- Conception et développement de l'authentification JWT et de la gestion des accès API ;
- Développement de la gestion des profils joueurs (création, mise à jour, préférences) ;
- Implémentation de la gestion du catalogue de jeux (niveau, rang, informations associées) ;
- Développement d'endpoints FastAPI pour le matching entre joueurs ;
- Participation au frontend React/Tailwind pour les parcours utilisateur principaux.

Méthodologie de travail mise en œuvre

- Analyse du besoin et définition du périmètre fonctionnel ;
- Découpage en couches (front, API, base de données) ;
- Développement itératif des fonctionnalités prioritaires ;
- Tests progressifs (API + parcours utilisateur) ;
- Versionnement Git et corrections successives selon les retours de test.

Architecture de la solution

- Frontend : React + Tailwind CSS pour l'interface et l'expérience utilisateur ;
- Backend : FastAPI pour les services métiers et les endpoints REST ;
- Données : MySQL pour le stockage des comptes, profils, jeux, matchs et messages ;
- Communication : échanges HTTP JSON entre le frontend et l'API.

Modalités de déploiement

Le projet est déployé en environnement local de développement/test, avec séparation front/API/BDD, configuration des accès applicatifs, scripts de lancement et validation fonctionnelle avant livraison.

Déploiement

Séparation frontend React/Vite, API FastAPI et base MySQL ; build et vérification avant livraison.

Sécurité

Authentification JWT, hachage bcrypt, validation Pydantic et contrôle des accès aux routes privées.

Test

Tests endpoints (avec Postman), tests de parcours front (inscription, profil, matching, messagerie), vérifications BDD.

Productions réalisées

- Conception de l'architecture globale de l'application ;
- Modélisation de la base de données ;
- Développement des routes API et de la logique métier ;
- Développement de l'interface utilisateur ;
- Intégration, tests et documentation.

Flux de traitement (étapes) :

- Inscription / connexion utilisateur
- Création et mise à jour du profil

- Sélection des jeux et niveaux
- Recherche de profils compatibles via l'API
- Communication entre utilisateurs
- Mise à jour des données en base MySQL